CS 213, 2001년 가을학기

# Malloc Lab: 동적 저장소 할당기 작성하기

배부: 11월 2일 금요일, 제출: 11월 20일 화요일 오후 11:59

Cory Williams (cgw@andrew.cmu.edu)가 이 과제의 담당자입니다.

## 1 소개

이 실습에서는 C 프로그램을 위한 동적 저장소 할당기, 즉 `malloc`, `free`, `realloc` 루틴의 여러분만의 버전을 작성하게 됩니다. 설계 공간을 창의적으로 탐색하여 올바르고 효율적이며 빠른 할당기를 구현해 보세요.

## 2 진행 안내

최대 2명까지 팀을 이루어 작업할 수 있습니다. 과제에 대한 공지, 정정, 추가 설명은 강의 웹페이지에 게시됩니다.

## 3 배포 파일 안내

**사이트별 안내: 여기에 학생들이 `malloclab-handout.tar` 파일을 어떻게 내려받는지 설명하는 문단을 넣으세요.**

먼저 작업할 안전한 디렉터리에 `malloclab-handout.tar`를 복사하세요. 그다음 `tar xvf malloclab-handout.tar` 명령을 실행합니다. 그러면 여러 파일이 해당 디렉터리에 풀립니다. 여러분이 수정해서 제출할 파일은 `mm.c` 하나뿐입니다. `mdriver.c` 프로그램은 여러분의 해법 성능을 평가할 수 있게 해주는 드라이버입니다. `make` 명령으로 드라이버를 만들고 `./mdriver -V` 명령으로 실행하세요. (`-V` 플래그는 유용한 요약 정보를 보여 줍니다.)

`mm.c` 파일을 보면 `team`이라는 C 구조체가 있는데, 여기에 팀원 1명 또는 2명의 신원 정보를 입력해야 합니다. 잊지 않도록 바로 작성하세요.

실습을 마치면 해답이 담긴 파일 `mm.c` 하나만 제출합니다.

## 4 실습 진행 방법

여러분의 동적 저장소 할당기는 아래 네 개 함수로 구성됩니다. 이 함수들은 `mm.h`에 선언되어 있고 `mm.c`에 정의되어 있습니다.

```
int mm_init(void);
void *mm_malloc(size_t size);
void mm_free(void *ptr);
void *mm_realloc(void *ptr, size_t size);
```

우리가 제공한 `mm.c` 파일은 생각할 수 있는 가장 단순하지만 기능적으로는 올바른 malloc 패키지를 구현합니다. 이를 시작점으로 삼아 이 함수들(그리고 필요하다면 private static 함수들)을 수정하여 다음 의미 규약을 만족하게 하세요:

- `mm_init`: 응용 프로그램(즉, 여러분의 구현을 평가할 때 사용할 트레이스 기반 드라이버 프로그램)은 `mm_malloc`, `mm_realloc`, `mm_free`를 호출하기 전에 `mm_init`을 호출하여 초기 힙 영역을 할당하는 등의 필요한 초기화를 수행합니다. 초기화에 문제가 있으면 -1, 그렇지 않으면 0을 반환해야 합니다.
- `mm_malloc`: `mm_malloc` 루틴은 최소 `size` 바이트 크기의 할당된 블록 payload를 가리키는 포인터를 반환합니다. 할당된 블록 전체는 힙 영역 안에 있어야 하며 다른 할당 블록과 겹치면 안 됩니다.

  여러분의 구현은 표준 C 라이브러리(libc)가 제공하는 `malloc`과 비교됩니다. `libc malloc`은 항상 8바이트 정렬된 payload 포인터를 반환하므로, 여러분의 구현도 항상 8바이트 정렬 포인터를 반환해야 합니다.
- `mm_free`: `mm_free` 루틴은 `ptr`이 가리키는 블록을 해제합니다. 반환값은 없습니다. 이 루틴은 전달된 포인터 `ptr`이 이전의 `mm_malloc` 또는 `mm_realloc` 호출로부터 반환되었고 아직 해제되지 않았을 때만 올바르게 동작함이 보장됩니다.
- `mm_realloc`: `mm_realloc` 루틴은 다음 제약을 만족하면서 최소 `size` 바이트 크기의 할당된 영역을 가리키는 포인터를 반환합니다.
  - `ptr`이 NULL이면, 이 호출은 `mm_malloc(size)`와 동일합니다.
  - `size`가 0이면, 이 호출은 `mm_free(ptr)`와 동일합니다.
  - `ptr`이 NULL이 아니라면, 이전의 `mm_malloc` 또는 `mm_realloc` 호출로부터 반환된 값이어야 합니다.

  `mm_realloc` 호출은 `ptr`이 가리키는 메모리 블록(기존 블록)의 크기를 `size` 바이트로 바꾸고 새 블록의 주소를 반환합니다. 구현 방식, 기존 블록의 내부 단편화 정도, 그리고 요청된 크기에 따라 새 블록의 주소는 기존 블록과 같을 수도 있고 다를 수도 있습니다.

새 블록의 내용은 기존 `ptr` 블록의 내용과 동일하되, 그 범위는 이전 크기와 새 크기 중 작은 값까지입니다. 나머지 바이트는 초기화되지 않습니다. 예를 들어 기존 블록이 8바이트이고 새 블록이 12바이트라면, 새 블록의 처음 8바이트는 기존 블록의 처음 8바이트와 동일하고 마지막 4바이트는 초기화되지 않습니다. 반대로 기존 블록이 8바이트이고 새 블록이 4바이트라면, 새 블록의 내용은 기존 블록의 처음 4바이트와 동일합니다.

이 의미 규약은 대응되는 libc의 `malloc`, `realloc`, `free` 루틴과 동일합니다. 전체 문서는 셸에서 `man malloc`을 실행해 확인하세요.

## 5 힙 일관성 검사기

동적 메모리 할당기는 올바르고 효율적으로 구현하기가 악명 높을 만큼 까다롭습니다. 타입 정보가 없는 포인터 조작이 많이 들어가기 때문에 실수 없이 작성하기가 어렵습니다. 힙을 훑으며 일관성을 검사하는 힙 검사기를 작성하면 큰 도움이 됩니다.

예를 들어 힙 검사기는 다음과 같은 항목을 검사할 수 있습니다:

- 가용 리스트의 모든 블록이 free로 표시되어 있는가?
- 서로 인접한 free 블록 중 병합(coalescing)되지 않은 블록이 있는가?
- 모든 free 블록이 실제로 가용 리스트 안에 들어 있는가?
- 가용 리스트의 포인터들이 유효한 free 블록을 가리키는가?
- 서로 겹치는 할당 블록이 존재하는가?
- 힙 블록 안의 포인터가 유효한 힙 주소를 가리키는가?

여러분의 힙 검사기는 `mm.c` 안의 `int mm_check(void)` 함수로 구성됩니다. 이 함수는 여러분이 필요하다고 생각하는 불변식이나 일관성 조건을 검사합니다. 힙이 일관적일 때에만 0이 아닌 값을 반환해야 합니다. 위에 제시된 항목만 검사해야 하는 것도 아니고, 모두 검사해야 하는 것도 아닙니다. `mm_check`가 실패했을 때 오류 메시지를 출력하도록 하는 것도 권장합니다.

이 검사기는 개발 중 디버깅을 위한 것입니다. `mm.c`를 제출할 때는 `mm_check` 호출을 모두 제거하세요. 처리량이 느려질 수 있기 때문입니다. `mm_check` 함수의 주석과 문서화 수준은 스타일 점수에도 반영됩니다.

## 6 지원 루틴

`memlib.c` 패키지는 여러분의 동적 메모리 할당기를 위해 메모리 시스템을 흉내 냅니다. `memlib.c`의 다음 함수를 사용할 수 있습니다:

- `void *mem_sbrk(int incr)`: `incr` 바이트만큼 힙을 확장하고, 새로 할당된 힙 영역의 첫 바이트를 가리키는 일반 포인터를 반환합니다. 의미는 Unix의 `sbrk` 함수와 동일하지만, `mem_sbrk`는 양의 0이 아닌 정수만 인자로 받습니다.

- `void *mem_heap_lo(void)`: 힙의 첫 바이트를 가리키는 일반 포인터를 반환합니다.
- `void *mem_heap_hi(void)`: 힙의 마지막 바이트를 가리키는 일반 포인터를 반환합니다.
- `size_t mem_heapsize(void)`: 현재 힙 크기를 바이트 단위로 반환합니다.
- `size_t mem_pagesize(void)`: 시스템의 페이지 크기를 바이트 단위로 반환합니다. (Linux에서는 4KB)

## 7 트레이스 기반 드라이버 프로그램

`malloclab-handout.tar`에 포함된 드라이버 프로그램 `mdriver.c`는 여러분의 `mm.c` 패키지를 정확성, 공간 활용도, 처리량 기준으로 검사합니다. 이 드라이버는 배포본에 함께 포함된 trace 파일들의 집합에 의해 제어됩니다. 각 trace 파일에는 할당, 재할당, 해제 요청들이 순서대로 담겨 있으며, 드라이버는 이 지시에 따라 `mm_malloc`, `mm_realloc`, `mm_free`를 호출합니다. 여러분이 제출한 `mm.c`를 채점할 때도 바로 이 드라이버와 trace 파일들이 사용됩니다.

`mdriver.c`는 다음 명령행 인자를 지원합니다:

- `-t <tracedir>`: 기본 trace 파일 대신 `tracedir` 디렉터리에서 trace 파일을 찾습니다.
- `-f <tracefile>`: 기본 trace 집합 대신 특정 trace 파일 하나만 테스트합니다.
- `-h`: 명령행 인자 요약을 출력합니다.
- `-l`: 학생용 malloc 패키지와 함께 libc malloc도 실행하고 측정합니다.
- `-v`: 각 trace 파일의 성능 결과를 압축된 표 형태로 자세히 출력합니다.
- `-V`: 더 자세한 진단 정보를 출력합니다. 디버깅 시 어떤 trace 파일에서 실패하는지 파악하는 데 유용합니다.

## 8 프로그래밍 규칙

- `mm.c` 안의 어떤 인터페이스도 변경해서는 안 됩니다.
- 메모리 관리와 관련된 라이브러리 호출이나 시스템 호출을 사용해서는 안 됩니다. 즉, 여러분의 코드 안에서 `malloc`, `calloc`, `free`, `realloc`, `sbrk`, `brk` 및 그 변형들을 호출할 수 없습니다.
- `mm.c` 안에서 배열, 구조체, 트리, 리스트 같은 전역 또는 `static` 복합 자료구조를 정의할 수 없습니다. 다만 정수, 실수, 포인터 같은 전역 스칼라 변수는 허용됩니다.

- `libc malloc` 패키지는 8바이트 경계에 맞춰 정렬된 블록을 반환하므로, 여러분의 할당기 역시 항상 8바이트 경계에 정렬된 포인터를 반환해야 합니다. 드라이버가 이 요구 사항을 검사합니다.

## 9 평가 기준

규칙을 어기거나 코드에 버그가 있어 드라이버를 크래시시키면 0점을 받습니다. 그렇지 않다면 성적은 다음과 같이 계산됩니다:

- 정확성 (20점): 드라이버가 수행하는 정확성 테스트를 모두 통과하면 만점을 받습니다. 각 trace를 올바르게 처리한 정도에 따라 부분 점수가 주어집니다.
- 성능 (35점): 여러분의 해법은 다음 두 지표로 평가됩니다.
  - 공간 활용도: 드라이버가 실제로 사용 중인 총 메모리량(`mm_malloc` 또는 `mm_realloc`로 할당되었고 아직 `mm_free`로 해제되지 않은 양)과 여러분의 할당기가 사용한 힙 크기 사이의 최대 비율입니다. 최적 비율은 1입니다. 단편화를 줄이는 좋은 정책을 택해 이 비율을 최대한 1에 가깝게 만들어야 합니다.
  - 처리량: 1초당 완료한 연산 수의 평균입니다.

드라이버는 여러분의 할당기 성능을 다음과 같은 performance index `P`로 요약합니다. `P`는 공간 활용도와 처리량의 가중합입니다.

$$P = wU + (1 - w)\ min(1, T / Tlibc)$$

여기서 `U`는 공간 활용도, `T`는 처리량, `Tlibc`는 기본 trace에서 시스템의 `libc malloc` 처리량 추정치입니다. 성능 지수는 처리량보다 공간 활용도를 조금 더 우선하며 기본값은 `w = 0.6`입니다.

메모리와 CPU 사이클은 모두 값비싼 시스템 자원이므로, 이 공식은 공간 활용도와 처리량을 균형 있게 최적화하도록 유도하기 위해 사용됩니다. 이상적으로는 `P = 1`, 즉 100%에 도달할 수 있습니다. 각 지표가 기여할 수 있는 최대치는 각각 `w`, `1 - w`이므로 어느 한쪽만 극단적으로 최적화해서는 좋은 점수를 받기 어렵습니다.

- 스타일 (10점):
  - 코드는 함수들로 잘 분해되어 있어야 하며 전역 변수는 가능한 한 적게 사용해야 합니다.
  - 코드 맨 앞에는 free 블록과 allocated 블록의 구조, free list의 조직 방식, 그리고 할당기가 free list를 어떻게 조작하는지 설명하는 헤더 주석이 있어야 합니다. 각 함수 앞에도 그 함수가 무엇을 하는지 설명하는 헤더 주석이 있어야 합니다.

---

*1 Tlibc □□ □□□□ □□ □□(600 Kops/s)□□, □□ □□□ □□□□□ □□□ □ □□ □□□□.*

- 각 서브루틴에는 그 함수가 무엇을 하고 어떤 방식으로 동작하는지 설명하는 헤더 주석이 있어야 합니다.
- 여러분의 힙 일관성 검사기 `mm_check`는 충분히 꼼꼼하고 문서화가 잘 되어 있어야 합니다.

좋은 힙 일관성 검사기에 5점, 프로그램 구조와 주석에 5점이 배점됩니다.

## 10 제출 안내

**사이트별 안내: 여기에 학생들이**
**해답이 담긴 `mm.c` 파일을 어떻게 제출하는지 설명하는 문단을 넣으세요.**

## 11 힌트

- `mdriver -f` 옵션을 사용하세요. 처음 개발할 때는 작은 trace 파일을 사용하면 디버깅과 테스트가 쉬워집니다. 초기 디버깅용으로 `short1-bal.rep`, `short2-bal.rep` 두 파일이 포함되어 있습니다.
- `mdriver -v`와 `-V` 옵션을 활용하세요. `-v`는 각 trace 파일의 자세한 요약을 보여 주고, `-V`는 각 trace 파일을 읽는 시점까지 표시해 주므로 오류를 분리하는 데 도움이 됩니다.
- `gcc -g`로 컴파일하고 디버거를 사용하세요. 디버거는 범위를 벗어난 메모리 접근을 찾는 데 큰 도움이 됩니다.
- 교재에 나오는 malloc 구현의 모든 줄을 이해하세요. 교재에는 implicit free list 기반의 단순한 할당기 예제가 자세히 나옵니다. 그 구현을 완전히 이해하기 전에는 자기 할당기 작업을 시작하지 마세요.
- 포인터 산술은 C 전처리기 매크로로 감싸세요. 메모리 관리자에서의 포인터 산술은 캐스팅이 많이 필요해서 혼란스럽고 실수하기 쉽습니다. 포인터 연산용 매크로를 만들어 두면 복잡도를 크게 줄일 수 있습니다.
- 구현은 단계적으로 진행하세요. 앞의 9개 trace는 `malloc`과 `free`만 포함하고, 마지막 2개 trace는 `realloc`, `malloc`, `free`를 포함합니다. 먼저 `malloc`과 `free`를 앞의 9개 trace에서 정확하고 효율적으로 동작하게 만든 뒤, 그다음 `realloc` 구현에 집중하는 것을 권장합니다.
- 프로파일러를 사용해 보세요. 성능 최적화에는 `gprof`가 도움이 될 수 있습니다.
- 일찍 시작하세요! 몇 페이지 코드만으로도 효율적인 malloc 패키지를 만들 수는 있지만, 지금까지 여러분이 작성한 코드 중 가장 어렵고 정교한 코드가 될 가능성이 큽니다. 그러니 일찍 시작하고, 행운을 빕니다!